

System Design Document - Sales Center

A sales lead management tool for a UK dealership group: one place to see every incoming enquiry, work it, and prove it was worked. This document states the business case, the architecture, the decisions that were genuinely contested, the judgment calls about where the design would break, and how the whole thing gets built and released by one person.

Layer built Frontend, backend mocked at the network boundary

Companion docs SYSTEM-ANALYSIS.md + SYSTEM-DESIGN.md + README.md + docs/

CONTENTS

1. Business Context — problem, value, scope boundary

2. High-Level Architecture - context, data flow, components, data model, APIs

3. Technical Decisions & Trade-offs - stack, CI, A over B
4. System Design Judgment - bottlenecks, dependencies, fallbacks

5. Observability - correlation, telemetry, fault injection

6. GenAI in Design - proposals, verification, division of labour

7. Rollout - rollout strategy, resourcing

PART 1 Business Context

What this solves for the user, what it is worth to the business.

1.1 The problem being solved

A dealership's sales floor does not lose deals because nobody can sell. It loses them in the gap between an enquiry arriving and a human touching it. Keyloop's own published research puts numbers on that gap:

Published findings that shaped what this tool prioritises.

FINDING	WHAT IT MEANS ON THE SALES FLOOR	WHAT THE PRODUCT THEREFORE DOES
20–30%	of enquiries are lost simply because they were never logged or never followed up (OC&C 2024, cited by Keyloop)	The inbox opens as a triage surface , sorted by what is about to breach, and states out loud how many enquiries have never been touched
69%	of dealers cite rekeying and double entry across systems (OC&C 2024, cited by Keyloop)	One lead record, one timeline. Every derived number — the SLA state, the working minutes, the triage counts — is computed, never retyped
8–10 systems	used to close a single deal, costing 5–10 hours per person per week (keyloop.com/sales-hub)	The enquiry, its whole activity history and its pipeline stage live on one record , not three

For the sales executive

They open one screen and it tells them what to do next. The response clock counts **working minutes** against the showroom's actual opening hours — including Sunday trading and bank holidays — so an enquiry that arrives at 19:40 on a Saturday is not reported as catastrophically late at 09:00 on Monday. Logging a call takes one dialog and persists immediately; the SLA clock stops server-side, from the activity itself, not from a checkbox someone has to remember to tick.

For the sales manager

They get a pipeline board showing where every open deal actually is, and stage guards that refuse a move the process does not allow — so the board reflects the floor rather than a parallel spreadsheet. Reporting is deliberately *not* in this phase: the activity log is the primary record, and analytics built over a log people have not yet got into the habit of filling teaches managers to distrust the tool. Once the log is reliably full, the numbers can be derived from it without a reconciliation step.

For the business

VALUE	MECHANISM	HOW IT WOULD BE MEASURED IN PRODUCTION
Revenue recovered from unworked leads	Unworked count is the first thing on screen; SLA chips escalate before breach	Count of leads with zero customer contact after 24 working hours → target zero
Faster speed-to-lead	Working-minutes SLA per source; at-risk threshold at 25% of budget remaining	p50 / p90 minutes to first genuine contact, by source and by executive
Less admin, fewer errors	One record and one timeline; every derived figure computed, never rekeyed	Hours per executive per week on data entry; correction rate on logged activity
Auditable compliance	Immutable chronological activity log; marketing consent with lawful basis per channel	FCA / GDPR audit sampling passes without manual reconstruction

1.2 In scope and out of scope for this phase

In scope — built and verified

Each is demonstrable by a command or a click.

- ✓ **Lead Inbox**
Triage bar, saved views, search, two sort orders, SLA chips
- ✓ **Lead Details + chronological activity log**
Day-grouped timeline, anomaly surfacing, SLA explainer
- ✓ **Activity logging that persists**
Survives reload; proven by end-to-end test, not by assertion
- ✓ **Working-hours SLA engine**
Bank holidays, Sunday trading, both DST discontinuities
- ✓ **Pipeline state machine + Kanban board**
12 stages, guards that carry a remedy, keyboard-operable moves
- ✓ **Observability layer**
Correlation ids, structured logs, Web Vitals, in-app panel

Out of scope

- ✗ **Deal Builder and the finance engine**
- ✗ **Manager analytics dashboard**
- ✗ **Propensity scoring**
A ranking is only worth having after the basics are consistently logged - otherwise it ranks the gaps in the data.
- ✗ **Real backend and database**
Assessment is frontend-focused; mocked at the network boundary behind a real HTTP client, so this is a base-URL change
- ✗ **Authentication, SSO, role-based permissions**
- ✗ **E-signature and quote sharing**
- ✗ **Predictive ML analytics**

PART 2 High-Level Architecture

Five views: what the system talks to, how one user action moves through it, how the frontend is decomposed, what the data looks like, and the contract between the two sides.

2.1 System Context Diagram

The Sales Center is a browser single-page application. In this phase it is **self-contained**: every external system it will eventually depend on is intercepted inside the browser by an MSW v2 service worker, which is the single integration boundary. Nothing in the application code knows it is talking to a mock — the UI issues real `fetch` calls to real `/api/v1/...` URLs.



Figure 1 — System Context. The application never calls an external system directly; it calls its own REST contract, and the MSW worker stands in for everything beyond it. That is what makes the mock disposable: replacing it with a real gateway changes a base URL and deletes a directory, not the application.

What each external system supplies, and what happens without it

EXTERNAL SYSTEM	WHAT WE NEED FROM IT	BEHAVIOUR TODAY · DEGRADATION IF IT IS DOWN
DMS (Keyloop Fusion)	Vehicle stock, customer master record, order write-back	Seeded stock in the mock. Degrades to read-only: quote and save locally, queue write-back
OEM portals / marketplaces	Inbound enquiries with source attribution	Seed generator produces them. If a feed stops, existing leads are unaffected; intake stalls and must alarm
DVLA licence check	Confirmation a licence is valid before a test drive	Recorded as a dated activity with a 365-day validity window. Fallback is a manual check logged by hand
Messaging gateway	Sending email / SMS / WhatsApp to the customer	Contact is logged, not sent. Fallback is an outbox: record the intent, retry the send

2.2 Data flow

The sequence diagram below is Part A's third requirement (Activity Logging) because it is the path that touches every layer. The detail worth noticing is that **the SLA clock is stopped server-side**: the handler asks the domain whether this activity counts as genuine customer contact. The UI cannot talk its way out of a missed target, and the same answer is given whether the question comes from the screen or the API.

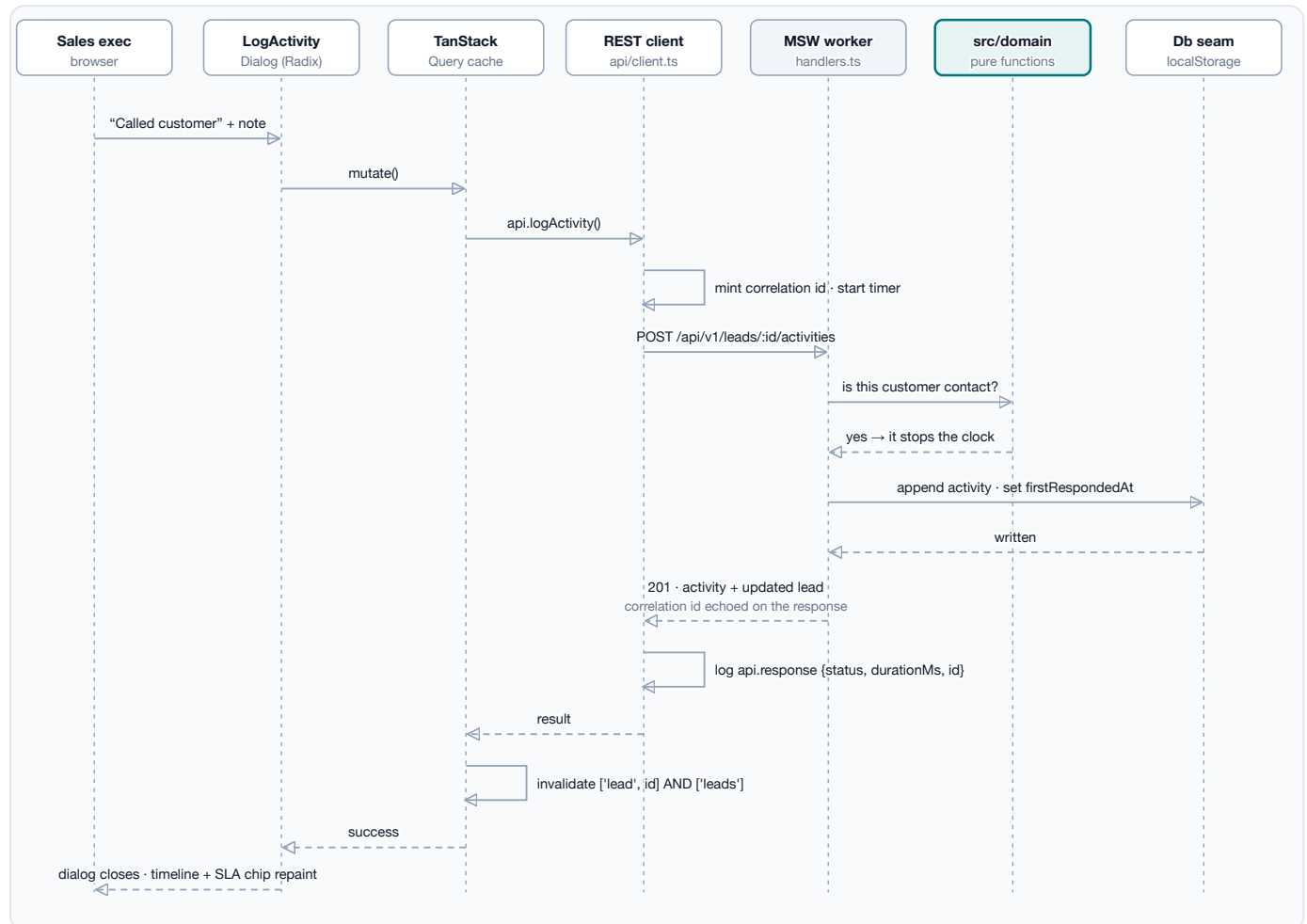


Figure 2 — Data flow for logging a follow-up activity. Invalidating *both* the lead and the inbox is deliberate: a genuine contact changes the lead's SLA state, so the inbox chip and the triage counts must re-derive too, not just the timeline. The correlation id minted at step 4 is echoed by the handler and appears on both log lines, so one click is traceable end to end from the in-app telemetry panel.

2.3 Frontend component design

The architecture is a **Layered (hexagonal)** shape adapted to a browser app. One rule holds it together:

THE ORGANISING CONSTRAINT

Every dealership rule lives in `src/domain/` as pure TypeScript — no React, no `fetch`, no DOM, and no ambient clock or randomness. React is a thin renderer of domain output. Time, ids and randomness enter only through injected ports (`Clock`, `IdGenerator`, `Rng`).

This is not a stylistic preference. It is what makes correctness demonstrable: the SLA engine, the working-hours calendar and the stage guards are all plain functions over plain data, unit-tested at 100% coverage — and *the same functions run inside the mock API*, so the rules cannot drift between what the screen believes and what the server enforces.

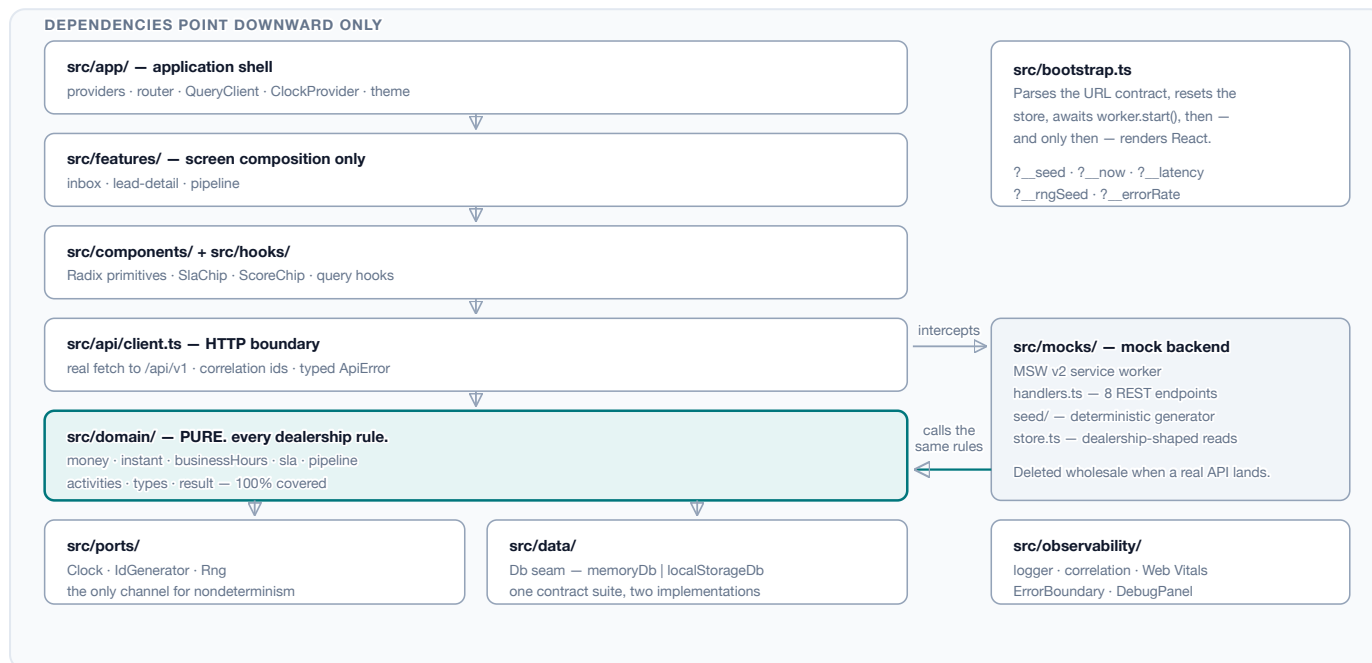


Figure 3 — Frontend module decomposition. The teal arrow is the point of the whole design: the mock API reaches *down* into the same pure domain layer the UI reads, so a stage guard or an SLA target has exactly one implementation. Purity is enforced mechanically — an ESLint `no-restricted-syntax` rule bans `Date.now()`, `new Date()` and `Math.random()` inside `src/domain`, and a test walks every file's imports to assert none escapes the directory.

Responsibilities, module by module

MODULE	RESPONSIBILITY	WHY IT IS A SEPARATE LAYER
<code>src/domain/</code>	Every dealership rule, as pure functions over plain data	Testable without a browser; reused verbatim by the mock server
<code>src/ports/</code>	<code>Clock</code> , <code>IdGenerator</code> , <code>Rng</code>	A single, auditable channel for nondeterminism
<code>src/data/</code>	The <code>Db</code> seam and two implementations	Makes “storage is swappable” a passing contract suite, not a claim
<code>src/mocks/</code>	MSW worker, handlers, deterministic seed generator	An executable specification of the API we need from the backend team
<code>src/api/</code>	Real HTTP client, real URLs, correlation ids, typed errors	The only module that knows HTTP exists — a real backend is a base-URL change
<code>src/app/</code>	Providers, routing, shell, and the one ticking clock	A single interval drives every countdown on screen

MODULE	RESPONSIBILITY	WHY IT IS A SEPARATE LAYER
<code>src/features/</code>	Screen composition only — no business logic	Thin by construction; the logic is always one layer below
<code>src/components/</code>	Radix-based primitives and shared display pieces	Accessibility solved once, not per screen
<code>src/observability/</code>	Structured logger, correlation, Web Vitals, error boundary, panel	Observability a reviewer can see without opening devtools

State ownership

Two owners, no overlap. **TanStack Query** owns all server state (leads, activities, reference data) — no server data is ever copied into another store. **The URL** owns shareable state (`?view=`, `?sort=`, and the `__`-prefixed bootstrap parameters). What is left — a sort toggle, the theme, whether the telemetry panel is open — is local component state, because a store for three booleans is ceremony rather than architecture. The payoff beyond tidiness is that Query's `useIsFetching()` and `useIsMutating()` give the end-to-end suite a real quiescence signal — which is why there is no `cy.wait()` anywhere in the Cypress suite.

2.4 Data model (Optional)

The model is deliberately small and lead-centric: **one aggregate root** (the lead), **one append-only child collection** (the activity log), **two reference collections** (sites and sales executives), and **one singleton** holding the scenario clock. Everything else — the customer, the vehicle, the part-exchange, the consent records — is an embedded value object living inside the lead document rather than a table of its own.

That shape follows from what the tool is. A dealership enquiry is a *conversation about one person and one car*, and the systems of record for customers and stock belong to the DMS. Embedding them keeps the enquiry self-contained and, more usefully, makes the vehicle of interest a **snapshot as recorded** rather than a pointer into live stock — so a later change in the stock system cannot silently rewrite what was discussed last week.

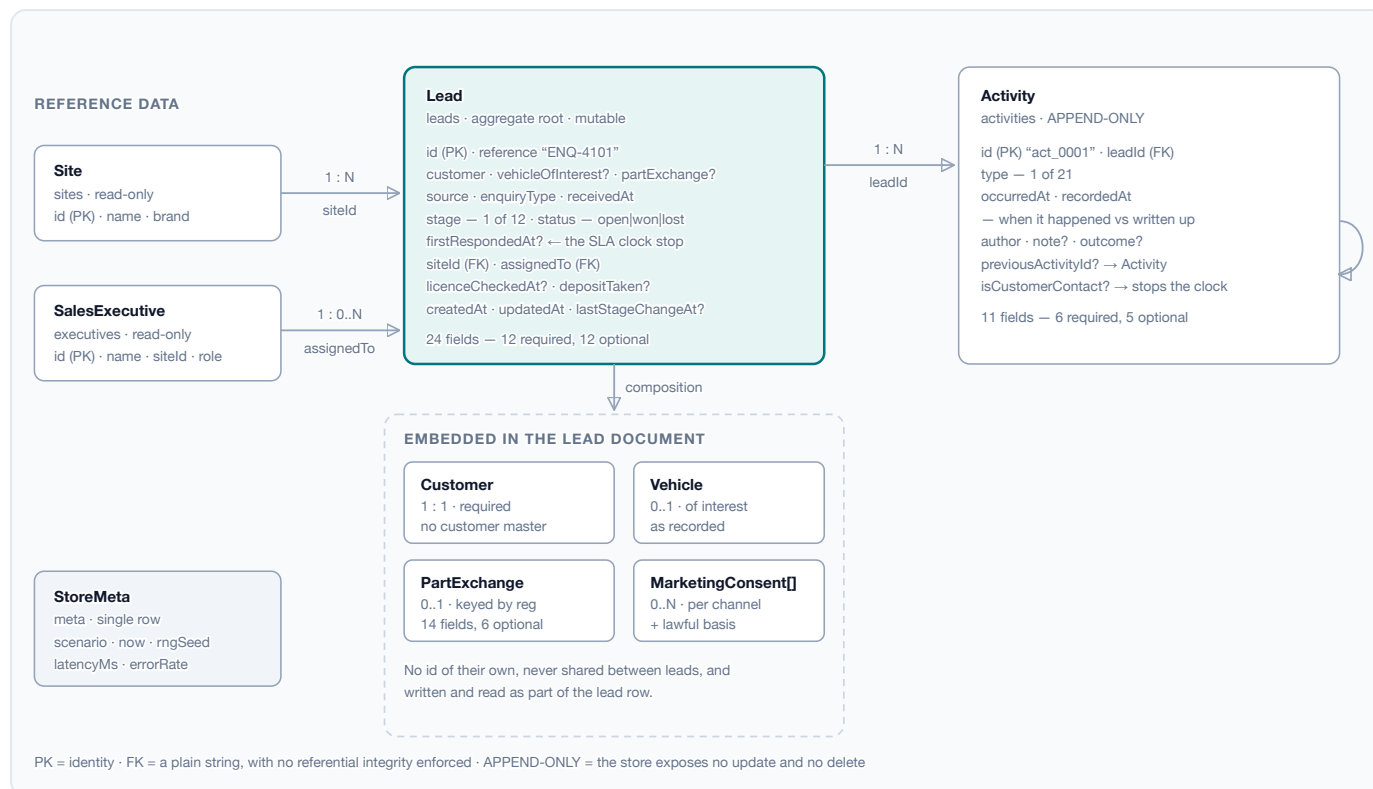


Figure 4 — Entity relationships. Five collections in total. The curve on **Activity** is the self-link that chains a timeline together, which is how Keyloop's own Sales Activities API models it — and it is what lets the timeline *detect* contradictory imported data instead of silently re-sorting it.

2.5 APIs and interfaces (Optional)

The mock backend exposes **eight REST endpoints** under `/api/v1`. They are not a convenience layer for the UI — they are the **contract the real backend has to honour**, written as executable code so the backend team can test against it rather than interpret prose. The application issues ordinary `fetch` calls to these paths; only the base URL would change.

ENDPOINT	WHAT IT DOES · DOMAIN FUNCTION	SUCCESS	FAILURE
GET <code>/reference</code>	Sites, executives and the scenario meta — one call at start-up	200 <code>{sites, executives, meta}</code>	503
GET <code>/leads</code>	The inbox. Filters on <code>status</code> , <code>siteId</code> and <code>assignedTo</code>	200 <code>{leads, total}</code>	503
GET <code>/leads/:leadId</code>	The lead record and its whole timeline in one payload	200 <code>{lead, activities}</code>	404, 503
GET <code>/leads/:leadId/activities</code>	The timeline alone	200 <code>{activities}</code>	404, 503
POST <code>/leads/:leadId/activities</code>	Logs an activity and may stop the SLA clock , decided by <code>isCustomerContact()</code>	201 <code>{activity, lead}</code>	404, 422 <code>INVALID_ACTIVITY_TYPE</code> , 503

ENDPOINT	WHAT IT DOES · DOMAIN FUNCTION	SUCCESS	FAILURE
PATCH /leads/:leadId/stage	Moves the lead — calls the same <code>moveStage()</code> the optimistic UI called	200 {lead}	404, 422 UNKNOWN_STAGE or any guard, 503
POST /leads/:leadId/close	Won or lost — <code>markWon()</code> / <code>markLost()</code>	200 {lead}	404, 422 LOST_REASON_REQUIRED , HANDOVER_REQUIRED , LEAD_CLOSED , 503
POST /leads/:leadId/reopen	<code>reopenLead()</code>	200 {lead}	404, 422 LEAD_ALREADY_OPEN , 503

Envelope conventions

Every success response is a **named-key object**, **never a bare array**, so a field can be added without breaking a client. Every failure is the same envelope —

`{ error: { code, message, remedy, details? } }` — and the `remedy` is not decoration: the UI renders it verbatim as the label on a one-click fix, and the identical string is asserted by both a unit test and an E2E test. Only **200, 201, 404, 422 and 503** are ever returned.

Error codes, and where each one is minted

The split below is the API-level evidence for the architecture: the handler validates the *request*, and every judgement about the *business* is passed through verbatim from the pure domain layer.

FAMILY	MINTED IN	CODES
Lookup and transport	handler	NOT_FOUND (404) · UPSTREAM_UNAVAILABLE (503, injected)
Request validation	handler	INVALID_ACTIVITY_TYPE · UNKNOWN_STAGE · LOST_REASON_REQUIRED
Pipeline rules	domain, passed through	LEAD_CLOSED · SAME_STAGE · BACKWARD_LIMIT_EXCEEDED · HANDOVER_REQUIRED · LEAD_ALREADY_OPEN
Stage guards	domain, passed through	LICENCE_CHECK_REQUIRED · DEPOSIT_REQUIRED · PREPARATION_REQUIRED_BEFORE_HANDOVER — each carries the remedy the UI turns into a button
Client-side only	api/client.ts	NETWORK_ERROR — synthesised when <code>fetch</code> itself rejects, so a dead network reaches the UI in the same shape as a refusal

PART 3 Technical Decisions & Trade-offs

The stack, then the decisions that were genuinely contested — each stated as what was chosen, what was rejected, and what the choice costs.

3.1 Frontend stack

LAYER	CHOICE	REASON
Framework	React 19	Mandated by the brief; the concurrent renderer and <code>useSyncExternalStore</code> are both used directly
Build tool	Vite 8	An internal tool behind a login has no SEO or SSR requirement to pay for
Language	TypeScript 6.0.3 , maximum strictness	<code>noUncheckedIndexedAccess</code> and <code>exactOptionalPropertyTypes</code> catch precisely the bugs that money and time code attract
UI library	Radix UI 1.6 (unified package)	Mandated. Focus traps, escape handling and ARIA wiring are easy to get subtly wrong and expensive to get wrong in an accessibility audit
CSS	Tailwind CSS v4 , CSS-first config	Mandated. v4 needs no config file or content globs; tokens live in <code>@theme</code> in one stylesheet
Server state	TanStack Query 5	Caching and in-flight tracking is where hand-rolled data layers rot; also supplies the E2E quiescence signal
Routing	React Router 8	URL-owned shareable state, including the deep-linkable demo scenarios

3.2 Backend stack (mocked this phase)

CONCERN	CHOICE	REASON
API mock	MSW v2 — the only mocking layer	Intercepts at the network boundary, so the app makes real <code>fetch</code> calls to real URLs. One handler set serves dev, Vitest and Cypress
API surface	8 REST endpoints under <code>/api/v1</code>	Leads, activities, stage, close, reopen, reference data — an executable contract the backend team can build against
Business rules	The same <code>src/domain</code> functions the UI uses	<code>PATCH /stage</code> calls the same <code>moveStage()</code> the optimistic UI called and returns 422 with the identical guard error — you can watch an optimistic move roll back on screen when the server refuses it
Persistence	<code>localStorage</code> behind a <code>Db</code> interface	Satisfies “activity logging must persist” with a seam that is contract-tested against a second in-memory implementation
Fixtures	Deterministic seed generator , seeded RNG	Fixed anchor leads for assertions plus generated volume for a credible-looking dealership — reproducible byte for byte
Fault injection	<code>?__latency</code> · <code>?</code> <code>__errorRate</code>	Failure and slowness are demonstrable on demand rather than described

3.3 Trade-offs — why A and not B

A trade-off is only real if the cost is named. Each row below states what was given up.

The decisions that had a credible alternative.

CHOSE	OVER	BECAUSE	WHAT IT COSTS US
React + Vite	Next.js	An internal tool behind a login has no SEO requirement and no anonymous traffic. SSR would add a server to operate, a hydration model to debug and a deployment target to secure — for zero user-visible benefit	No server-side data fetching or streaming; we ship a client bundle and pay for route-level code splitting ourselves
Pure domain layer + ports	Logic inside components and hooks	Three of the four graded dimensions reward evidence. Pure functions can be proven at 100% coverage and reused by the mock server; logic inside a component can only be demonstrated by clicking	More files and one more indirection; the purity rule must be enforced by lint and a test, because discipline alone would not hold
MSW v2 service worker	json-server or an Express mock	The app issues real <code>fetch</code> to real URLs, so swapping in a real backend is a base-URL change. One handler set serves dev, unit tests and E2E — a second process would mean a second, drifting definition of the API	A real cost: <code>cy.request</code> runs in Cypress's Node process and never reaches a service worker, so test state could not be reset over HTTP. That forced a URL bootstrap contract (<code>?__seed</code> , <code>?__now</code>) — which turned out to be better anyway, since every scenario is now a shareable link
TanStack Query, and nothing else	Redux Toolkit, or a client store alongside it	Almost all state here is server state. Query gives caching, invalidation and in-flight tracking for free, and <code>useIsFetching()</code> is what lets the E2E suite wait on a real signal instead of a timer. What remained was small enough to live in component state	No shared client store, so genuinely cross-screen UI state would need one adding
Integer pence + branded types	Floating point, or Decimal.js	Any chain of monetary operations drifts a penny at every step once floats are involved, and the error compounds. Integers are exact and need no dependency	Conversion ceremony at every UI edge, and a branded type that newcomers must learn
Radix primitives + Tailwind	MUI, Chakra or a full component kit	Mandated, and correct anyway: unstyled primitives give correct focus and ARIA behaviour without a theme to fight. A dense operational table is exactly what opinionated kits make hard	We build the visual layer ourselves — more CSS, and the colour palette had to be computed for contrast rather than picked
Keyboard-first Kanban	Pointer drag-and-drop	No drag-and-drop library was in the dependency set, the keyboard path is accessible by default, and it is deterministically testable — pointer DnD is the single flakiest thing to automate	A less impressive first impression in a demo

CHOSE	OVER	BECAUSE	WHAT IT COSTS US
Vitest 4 · Cypress 15 · jsdom 29	The latest major of each	The development machine runs Node 25 (odd, non-LTS). Cypress 16, Vitest 5 and jsdom 30 all exclude it in their <code>engines</code> . The pinned line runs on Node 20.1 / 22 / 24 / 25 / 26 alike — so it also runs on whatever the reviewer has	Not on the newest line. The gain: the whole set installs with zero overrides and zero <code>--legacy-peer-deps</code>
<code>retry: false</code> on queries	Automatic retry with backoff	The mock fails <i>on purpose</i> , for the reliability demo. Retrying would hide exactly the behaviour being demonstrated	Correct for a demo, wrong for production — §4.3 states what replaces it

PART 4 System Design Judgment

Where this breaks, what it depends on, and what it does when something it depends on goes away.

4.1 Bottlenecks — what fails first under a spike

Today every request is served inside the browser, so there is no server to overload. The load that matters is therefore **data volume per user**: a group with fifteen sites and a busy plate-change month. The table below names the real limits in the code as written, in the order they would bite.

PRESSURE POINT	RISK	WHAT ACTUALLY BREAKS	MITIGATION
<code>localStorage</code> as the store	First to break	Synchronous, main-thread, ~5 MB per origin, and it serialises the whole collection on every write. At a few thousand leads the write cost becomes a visible jank on every activity logged; past the quota it throws	It is already a seam. <code>localStorageDb</code> degrades to memory on failure today; the production path is IndexedDB or, properly, a real API — one class, no caller changes
<code>GET /leads</code> returns everything	First to break	No pagination: every matching lead comes back in one payload, and <code>total</code> merely echoes the array length rather than describing a page window. The client then sorts and filters the whole set on the main thread	Cursor pagination with the sort applied server-side, so the payload is bounded by the page rather than by the dealership. Removing propensity scoring already took an O(leads × activities) loop out of this handler
One-second clock tick × every SLA chip	Large inbox	<code>ClockProvider</code> ticks at 1 s and every countdown re-renders. With an unvirtualised list of a thousand rows that is a thousand re-renders per second	Virtualise the list; drop the tick to 30 s for targets more than an hour out. The clock is already a single injected port, so this is one change in one place
Working-minutes calculation	Bounded	The business-hours walk iterates day by day, capped at 400 iterations. Cheap per call, but it runs per lead per tick	Memoise per <code>(receivedAt, source)</code> pair; the function is pure, so caching is safe by construction
Service worker boot	Dev only	Every request passes through MSW. It is a fixed per-request cost that disappears entirely in production	None needed — the mock is deleted when a real API lands

4.2 Dependency risk

This phase was deliberately built so that **no other team can block it**. That is the purpose of mocking at the network boundary rather than waiting for an API. The risks that remain are about the *handover*, not the build.

DEPENDENCY	RISK	EXPOSURE	HOW IT IS DE-RISKED
Backend / data team — the real lead and activity API	Medium	The largest one. If their contract differs from ours, integration is rework rather than a swap	The MSW handlers are the contract — 8 endpoints with request and response shapes, error codes and status mapping, all executable. The backend team gets a specification they can test against instead of a conversation. Blast radius of a mismatch is one adapter module, <code>src/api/client.ts</code>
Identity / SSO team	Low	No authentication in this phase; the client holds no token logic at all	Every request already goes through one client module, so auth arrives as a request interceptor plus a 401 handler. Nothing else changes
DMS team (Keyloop Fusion)	Medium	Stock, customer master and order write-back. Write-back is the genuinely hard part — it is someone else's system of record	Reads are cacheable and degrade to stale. Writes should be queued and idempotent from the start; the activity log already gives an audit trail to reconcile against
DVLA and the messaging gateway	Low	Each is a third party with its own availability	Each has a manual fallback that is already the current process: log the licence check by hand, record the contact as an activity
Library vendors	Low	React, Radix, TanStack Query, Tailwind, MSW — all MIT and widely adopted. MSW is dev-only, so it carries no production risk	The one with real lock-in is TanStack Query, because it shapes the data layer. Accepted deliberately: it is the most stable piece of the set
Node 25 on the build machine	Resolved	An odd-numbered, non-LTS runtime that several current majors exclude	Versions pinned to the widest-compatible line; CI runs a Node 22 / 24 / 25 matrix so a regression on any of them fails the build

4.3 Fallback plan

The question is whether a failure surfaces an error or recovers. The honest answer differs by failure, and differs between this phase and production — so both are stated.

FAILURE	BEHAVIOUR TODAY	PRODUCTION BEHAVIOUR
Read request fails (5xx / network)	A typed <code>ApiError</code> surfaces an error panel carrying the failure's <code>remedy</code> string and its correlation id. No retry — deliberate, so the fault-injection demo is honest	Retry idempotent GETs three times with exponential backoff and jitter, then fall back to stale-while-error : render the last good cached data behind a “showing data from 09:14” banner. A stale inbox beats an empty one
Write fails (logging an activity)	The mutation surfaces the error and the dialog stays open with the user's text intact — the one thing that must never be lost is what the executive just typed	Attach an idempotency key per activity, queue the write in an outbox, retry in the background, and reconcile on reconnect. Never silently drop

FAILURE	BEHAVIOUR TODAY	PRODUCTION BEHAVIOUR
Business rule rejects the action	Not a failure — a guard. The API returns 422 with <code>{code, message, remedy}</code> , and the UI renders the remedy as a one-click fix (“Log licence check”). The same string is asserted by both the unit test and the E2E test	Unchanged. This is the designed behaviour
<code>localStorage</code> unavailable or over quota	Already handled. <code>localStorageDb</code> degrades to an in-memory store, warns once, and the application keeps working — private browsing and full-disk are both covered	Same pattern against IndexedDB, with the outbox above so nothing is lost when storage is the thing that failed
Third party down (lender, valuation, DVLA, messaging)	Not called in this phase	Degrade, never block. Quote locally and queue the proposal (“submitted — awaiting lender”); fall back to manual entry for valuations; record the contact intent in an outbox when the gateway is unreachable. Circuit-break per partner so one slow integration cannot exhaust the connection pool
A component throws while rendering	An error boundary catches it, logs <code>ui.render_error</code> with the component stack, and shows a recoverable panel instead of a white screen	Same, with the log line shipped to the collector — the correlation id already links it to the request that preceded it
Service worker fails to register	Known gap. <code>main.tsx</code> awaits <code>worker.start()</code> before rendering and does not catch a rejection, so a failure to register the worker would leave a blank page rather than an explanation	Moot in production — there is no worker. For the demo build the fix is small and worth making: catch the rejection and render a static “mock backend failed to start” panel

4.4 How these claims can be checked

Every statement in this document is backed by something executable. From the project root:

COMMAND	WHAT IT DEMONSTRATES
<code>npm run verify</code>	Typecheck, lint, determinism, 475 unit tests with a 100% gate on <code>src/domain</code> , and a production build
<code>npm run e2e</code>	58 Cypress specs across seven journeys, zero retries, no <code>cy.wait()</code> anywhere
<code>npm run mutation-sanity</code>	Four domain constants are flipped and the suite must go red — proof the tests constrain behaviour rather than merely execute it

PART 5 Observability

LAYER	IMPLEMENTATION	WHERE YOU SEE IT
Correlation ids	Minted per request, echoed by every handler	On the request and response log lines
Structured logging	<code>{level, msg, at, correlationId, ...};</code> warn/error as one-line JSON	<code>docs/observability.md</code>
Ring buffer + panel	Last 200 records via <code>useSyncExternalStore</code>	Telemetry button, or <code>Ctrl</code> + <code>`</code>
Web Vitals	LCP, INP, CLS, FCP, TTFB into the same log	Panel, <code>web_vital</code> entries
Error boundaries	Log <code>ui.render_error</code> with the component stack	<code>ErrorBoundary.test.tsx</code>
Fault injection	<code>?__latency=400&__errorRate=0.25</code>	Every Nth write fails - reproducible

Ids are **sequential, not** `crypto.randomUUID()`, so a Cypress run is reproducible and a spec can assert on a specific trace.

In production this is a transport swap: `logger`'s `emit` posts to an OTLP collector or Sentry, and the correlation id becomes the trace id that joins the browser span to the backend span.

PART 6 How GenAI was used in the design phase

The implementation narrative is in the README. This section covers **design**, where GenAI was used differently - and where its output required the most scrutiny.

The judges were instructed to recompute, not merely to opine

That produced concrete findings:

- They recomputed every worked finance example and found arithmetic or formula errors in all three proposals. One quoted a PCP payment of £328.14 where its own formula produced £367.75. This work informed verification discipline even though the finance enhancement was later removed from the MVP.
- One judge identified that a proposed E2E strategy depended on `cy.request` reaching MSW, which it cannot do because Cypress sends that request from its Node process. That finding became §3.2 and changed the application's startup sequence.
- A proposed frozen demonstration clock landed on a Sunday morning outside the configured opening hours, which would have flattened the SLA countdowns the demo needed to show.

Confident library claims were verified by execution

Two research findings proved false when the code ran:

`eslint-plugin-react-hooks@7.1.1`'s `recommended-latest` configuration still used the legacy ESLint shape, and `jsdom@30` excluded Node 25. A third assumption - that rendered `oklch()` contrast could be predicted reliably with a naive sRGB conversion - failed near the gamut edge. In that case, `cypress-axe` became the arbiter instead of the generated analysis.

The resulting division of labour was deliberate

GenAI was effective at breadth: enumerating UK motor-retail vocabulary, drafting realistic seed data and generating exhaustive test cases. It was less reliable for arithmetic and current library behaviour. The working rule therefore became: **let GenAI propose, then require an independent mechanism to adjudicate**. The mutation sanity check, accessibility gate, deterministic E2E contract and 144-pair pipeline table are retained examples of that rule. The independent IRR oracle was another example and was retired with the finance engine when the scope was reduced to the MVP.

PART 7 Rollout

Turning the design above into backlog items: how the work reaches users, who does it, and in what order it ships.

7.1 Rollout Strategy

This phase ships no production backend, so what follows is the plan for when the real API lands. It is recorded now rather than later because the shape of a rollout is a design decision — the cohort you expose first determines what the pilot can actually tell you.

Staged rollout. Each stage has an explicit promotion test rather than a judgement call.

STAGE	COHORT	PROMOTE WHEN — MEASURED FROM THE ACTIVITY LOG THE TOOL ALREADY KEEPS
0 · Dogfood	Me plus one sales manager, on seeded data	All seven E2E journeys pass on real showroom hardware and a tablet
1 · Pilot site	One dealership, all of its executives (~7% of users)	Speed-to-lead p90 no worse than the incumbent; zero leads unworked past 24 working hours; no P1 defect; activity-logging rate above the old system's
2 · One brand	Every site of a single franchise brand	Stage 1 metrics hold at roughly 5× the volume; support contacts per user flat or falling
3 · Group	All sites	Stage 2 metrics hold across every brand in the group

7.2 Resourcing

Resource allocation — one headcount

RESPONSIBILITY	OWNS	WHAT A SEPARATE PERSON WOULD NORMALLY CATCH
Frontend	Three screens, the component layer, accessibility, theming	A designer noticing an unreadable colour; a second developer noticing business logic leaking into a component
Backend (mocked)	Eight endpoints, the persistence seam, the seed generator, the error contract	A backend engineer refusing a contract that is awkward to implement for real
Automation testing	475 unit tests, 58 E2E specs, the coverage gate, the CI pipeline	A tester who did not write the code and therefore does not share its assumptions